

数据库设计说明书

一、引言

1.1 编写目的

本说明书旨在明确“情绪管理与 AI 共情交互系统”的数据库设计方案，规范数据存储结构、表间关联关系及数据流转逻辑，确保系统数据的完整性、一致性和可扩展性，为后端开发、数据存储及后续数据聚合分析提供技术依据。

1.2 项目背景

- 项目名称：**情绪管理与 AI 共情交互系统
- 项目目标：**通过接收用户提交的情绪日记、与 AI 角色的对话内容，实现情绪识别、共情回复生成及情绪轨迹可视化，辅助用户进行自我情绪认知与管理。
- 核心需求：**需存储用户信息、情绪日记、AI 角色、对话记录、情绪轨迹等数据，并建立关联关系以支持数据追溯与聚合分析。

1.3 定义与缩写

缩写	全称	说明
OpenID	开放平台用户唯一标识	微信小程序用户身份标识
OSS	对象存储服务	存储语音文件的云存储服务
AI	人工智能	用于情感分析与共情回复生成
ECharts	企业级图表库	前端可视化工具

二、数据库环境说明

- 数据库类型：**关系型数据库
- 数据库管理系统：**MySQL 8.0（主存储）+ Redis 7.0（缓存）
- 字符集：**UTF-8（支持多语言文本存储）
- 存储引擎：**InnoDB（支持事务与外键约束）
- 缓存策略：**Redis 缓存高频访问的角色参数、情绪标签，降低数据库查询压力

三、数据库命名规范

- 表名：**采用“模块名_子模块名”格式，全小写，下划线分隔（如`user\_info`、`emotion\_diary`）。
- 字段名：**全小写，下划线分隔，语义明确（如`user\_openid`、`diary\_content`）。
- 主键：**统一命名为`id`，类型为`BIGINT`，自增。
- 外键：**命名为`关联表名\_id`（如`user\_id`关联`user\_info.id`）。

- 索引：普通索引命名为`idx\_字段名`，唯一索引命名为`uk\_字段名`。

四、数据库 ER 图（概念设计）

erDiagram

user_info ||--o{ emotion_diary : "1 对多"

user_info ||--o{ role_chat : "1 对多"

user_info ||--o{ emotion_track : "1 对多"

ai_role ||--o{ role_chat : "1 对多"

emotion_diary ||--|| emotion_track : "1 对 1（按日聚合）"

user_info {

BIGINT id PK "用户 ID"

VARCHAR(64) user_openid UK "微信 OpenID"

VARCHAR(32) nickname "昵称"

DATETIME create_time "创建时间"

}

ai_role {

BIGINT id PK "角色 ID"

VARCHAR(32) role_name "角色名称"

DECIMAL(3,2) gentleness "温柔度(0-1)"

DECIMAL(3,2) rationality "理性值(0-1)"

}

emotion_diary {

BIGINT id PK "日记 ID"

BIGINT user_id FK "用户 ID"

TEXT diary_content "日记内容"

VARCHAR(64) voice_oss_path "语音 OSS 路径"

VARCHAR(255) emotion_tags "情绪标签"

TEXT ai_summary "AI 内容总结"

TEXT ai_reply "AI 共情回复"

DATETIME create_time "创建时间"

```
}

role_chat {
BIGINT id PK "对话 ID"
BIGINT user_id FK "用户 ID"
BIGINT role_id FK "角色 ID"
TEXT user_input "用户输入"
TEXT ai_reply "AI 回复"
DATETIME chat_time "对话时间"
}

emotion_track {
BIGINT id PK "轨迹 ID"
BIGINT user_id FK "用户 ID"
DATE track_date "日期"
INT anxiety_value "焦虑值"
INT joy_value "愉悦值"
VARCHAR(255) key_event "关键事件"
}
```

五、数据表结构设计（逻辑设计）

5.1 用户表（user_info）

存储微信小程序用户基本信息，通过`user\_openid`唯一标识用户。

字段名	数据类型	长度	允许空	主键	外键	默认值	说明
id	BIGINT	。	否	是	否	自增	用户 ID（主键）
user_openid	VARCHAR	64	否	否	否	。	微信 OpenID（唯一索引）
nickname	VARCHAR	32	是	否	否	NULL	用户昵称
create_time	DATETIME	。	否	否	否	CURRENT_TIME	创建时间

						STAMP	
--	--	--	--	--	--	-------	--

5.2 AI 角色表 (ai_role)

存储 AI 角色性格参数，用于共情回复风格控制。

字段名	数据类型	长度	允许空	主键	外键	默认值	说明
id	BIGINT	。	否	是	否	自增	角色 ID（主键）
role_name	VARCHAR	32	否	否	否	。	角色名称（如“暖心朋友”）
gentleness	DECIMAL	3,2	否	否	否	0.80	温柔度（0-1，越高越温柔）
rationality	DECIMAL	3,2	否	否	否	0.50	理性值（0-1，越高越理性）

5.3 情绪日记表 (emotion_diary)

存储用户提交的情绪日记内容及 AI 分析结果。

字段名	数据类型	长度	允许空	主键	外键	默认值	说明
id	BIGINT	。	否	是	否	自增	日记 ID（主键）
user_id	BIGINT	。	否	否	是 (user_info.id)	。	用户 ID（外键）
diary_content	TEXT	。	否	否	否	。	日记文本内容（语音转文字后结

							果)
voice_oss_path	VARCHAR	128	是	否	否	NULL	语音文件 OSS 存储路径（语音输入时非空）
emotion_tags	VARCHAR	255	是	否	否	NULL	情绪标签（如“焦虑-工作压力, 愉悦-聚会”）
ai_summary	TEXT	。	是	否	否	NULL	AI 生成的内容总结
ai_reply	TEXT	。	是	否	否	NULL	AI 共情回复文本
create_time	DATETIME	。	否	否	否	CURRENT_TIMESTAMP	日记创建时间

5.4 角色对话表 (role_chat)

存储用户与 AI 角色的对话记录。

字段名	数据类型	长度	允许空	主键	外键	默认值	说明
id	BIGINT	。	否	是	否	自增	对话 ID（主键）
user_id	BIGINT	。	否	否	是 (user_info.id)	。	用户 ID（外键）
role_id	BIGINT	。	否	否	是 (ai_role.id)	。	角色 ID（外键）

user_input	TEXT	◦	否	否	否	◦	用户输入的对话内容
ai_reply	TEXT	◦	否	否	否	◦	AI 生成的共情回复
chat_time	DATETIME	◦	否	否	否	CURRENT_TIMESTAMP	对话时间

5.5 情绪轨迹表 (emotion_track)

按日聚合用户情绪值，用于可视化分析。

字段名	数据类型	长度	允许空	主键	外键	默认值	说明
id	BIGINT	◦	否	是	否	自增	轨迹ID（主键）
user_id	BIGINT	◦	否	否	是 (user_info.id)	◦	用户ID（外键）
track_date	DATE	◦	否	否	否	◦	日期（如“2026-05-10”）
anxiety_value	INT	◦	否	否	否	0	焦虑值（0-100，基于情绪标签权重计算）
joy_value	INT	◦	否	否	否	0	愉悦值（0-100，基于情绪标签权重计算）
key_ev	VARCHAR	255	是	否	否	NULL	关键事

ent	R						件标记 (如 “5 月 因工作 压力情 绪低 谷”)
-----	---	--	--	--	--	--	---

六、视图设计

6.1 用户情绪日报视图 (v_user_daily_emotion)

整合用户每日情绪日记、对话及情绪值，支持前端快速查询。

```
CREATE VIEW v_user_daily_emotion AS
SELECT
u.user_openid,
d.track_date,
d.anxiety_value,
d.joy_value,
GROUP_CONCAT(DISTINCT ed.emotion_tags) AS all_emotion_tags,
GROUP_CONCAT(DISTINCT rc.ai_reply) AS recent_ai_replies
FROM user_info u
JOIN emotion_track d ON u.id = d.user_id
LEFT JOIN emotion_diary ed ON u.id = ed.user_id AND DATE(ed.create_time) =
d.track_date
LEFT JOIN role_chat rc ON u.id = rc.user_id AND DATE(rc.chat_time) =
d.track_date
GROUP BY u.user_openid, d.track_date;
```

七、存储过程与触发器

7.1 情绪值计算存储过程 (sp_calculate_emotion_value)

基于情绪标签权重，按日计算用户焦虑值、愉悦值，并插入`emotion\_track`表。

```
DELIMITER //
CREATE PROCEDURE sp_calculate_emotion_value(IN input_user_id BIGINT, IN
input_date DATE)
BEGIN
```

```

DECLARE v_anxiety INT DEFAULT 0;
DECLARE v_joy INT DEFAULT 0;
DECLARE v_key_event VARCHAR(255) DEFAULT NULL;
-- 计算焦虑值（“焦虑”标签权重 80，“压力”权重 70）
SELECT SUM(
CASE
WHEN emotion_tags LIKE '%焦虑%' THEN 80
WHEN emotion_tags LIKE '%压力%' THEN 70
ELSE 0
END
) INTO v_anxiety
FROM emotion_diary
WHERE user_id = input_user_id AND DATE(create_time) = input_date;
-- 计算愉悦值（“愉悦”标签权重 90，“开心”权重 80）
SELECT SUM(
CASE
WHEN emotion_tags LIKE '%愉悦%' THEN 90
WHEN emotion_tags LIKE '%开心%' THEN 80
ELSE 0
END
) INTO v_joy
FROM emotion_diary
WHERE user_id = input_user_id AND DATE(create_time) = input_date;
-- 标记关键事件（如焦虑值>150 时标记“情绪低谷”）
IF v_anxiety > 150 THEN
SET v_key_event = '情绪低谷（高焦虑）';
END IF;
-- 插入情绪轨迹表
INSERT INTO emotion_track (user_id, track_date, anxiety_value, joy_value,

```



```
key_event)
VALUES (input_user_id, input_date, v_anxiety, v_joy, v_key_event)
ON DUPLICATE KEY UPDATE
anxiety_value = v_anxiety,
joy_value = v_joy,
key_event = v_key_event;
END //
DELIMITER ;
```

八、安全性设计

- **权限控制：**
 - 应用账号 (app_user)：仅允许`SELECT`、`INSERT`、`UPDATE`操作，禁止`DELETE`（逻辑删除替代）。
 - 管理账号 (admin_user)：允许全权限操作，仅限内网访问。
- **数据加密：**
 - 敏感字段（如`user\_openid`）采用 AES 加密存储。
 - 语音文件 OSS 路径通过签名 URL 访问，避免未授权下载。
- **备份策略：**
 - 每日全量备份（凌晨 2 点），保留 7 天。
 - 每小时增量备份，保留 24 小时。

九、性能优化

- **索引优化：**
 - `emotion\_diary(user\_id, create\_time)`：加速用户日记按时间查询。
 - `role\_chat(user\_id, chat\_time)`：加速用户对话记录查询。
 - `emotion\_track(user\_id, track\_date)`：加速情绪轨迹按日期聚合。
- **缓存策略：**
 - Redis 缓存`ai\_role`表数据（缓存时间：1 小时），减少数据库查询。
 - 缓存用户最近 7 天情绪轨迹（键：`emotion\_track:user\_id:last\_7days`）。

十、附录

- **ER 图源文件：**`emotion\_system\_er.mwb`（MySQL Workbench 格式）
- **SQL 脚本：**`create\_tables.sql`、`create\_views.sql`、

`create\_procedures.sql`

- **参考文档:**《系统设计说明书》V1.0、《微信小程序开发文档》
(AI 生成)